bathy

A TCP discovery scanner for *authorized* network questions, whose primary interface is a typed MCP tool surface: eleven tools with published JSON Schemas, service findings that carry the digest of the bytes that produced them, and an authorization manifest an agent has no field in which to supply.

v0.1.0-alpha.2 · crates.io/crates/bathy · github.com/russell0/bathy · Apache-2.0 OR MIT · file:line references at git HEAD 33dce4d

WHAT BREAKS WHEN A MODEL DRIVES A SCANNER

Nmap is excellent at what it is for. This is an argument about a different consumer, and it states what the trade costs. Three failure modes, all properties of the interface rather than the model. A field that holds a command line gets filled with a generated one, and a sanitizer becomes the only thing between the loop and the network. If scope is a parameter, scope is negotiable. And if the output is a complete record, the agent pays for the whole record and authors the predicate itself.

That third one is measurable. In an `nmap -sV --open -oX` document from the lab, the two open TLS-wrapped :443 endpoints are spelled `name="http" tunnel="ssl" method="probed" conf="10"` and `name="https" tunnel="ssl" method="table" conf="3"`: the obvious XPath finds the first and misses the second; its mirror image does the reverse. No flag fixes that: the format is faithfully recording a real difference between a probed answer and a port-table guess, and the consumer makes the judgement. The rest of the mess is flag-dependent, and the document says so: the same target set scanned without `--open` is 143 `<port>` elements, 131 not open and every one still carrying a `<service>`, so that XPath returns 12 hits of which 2 are open, two being addresses the ground truth lists as `absent` — present only because `-Pn` was passed, and marked `filtered`. With `--open` it collapses to 2 hits, both open. Two thirds of the problem is a better XPath. The `:443` pair is not. → census over target/agentcost/nmap-version-open.xml and nmap-version.xml; lab/ground-truth.json:86

THREE DIFFERENCES, EACH WITH A TEST BEHIND IT

1. No input schema can express a command line

Eleven tools from one compiled-in constant, confirmed on the wire against `bathy serve mcp`. All eleven input schemas set `additionalProperties: false` via `#[serde(deny_unknown_fields)]`: an unrecognised argument is a parse refusal the model can see, not a field silently dropped. Three tests — over the live wire schema, the in-process descriptor, and the derived schema set `check-schemas` holds the 27 committed documents to — render each schema to a string, `$defs` included, and assert it contains none of `command`, `args`, `flags`, `argv`, `raw`. Structurally: `grep -rn 'Command::new' crates/bathy-mcp/src/` is empty, and the workspace's only production subprocess spawn is `packetd.rs:308`, which passes no arguments — the path it runs is a `PacketdConfig` field set by the host, absent from every request type. → descriptors.rs:36-48,417 · tests/mcp.rs:1102 · bathy-types/src/tools.rs:545, schema.rs:14 · bathy-engine/src/packetd.rs:62,77-90,308

2. The agent cannot author, widen, or self-grant its authorization through the tool surface

A manifest is twelve lines of JSON an operator writes: id, description, expiry, allowed and denied CIDRs, a packets/runtime/rate ceiling — 451 bytes in `lab/scope.json`. No input schema accepts an inline manifest or a scope id. The four scope-taking tools name a file by `manifest_path`, asserted both ways, so the test fails if a fifth gains the field or one of the four loses it. On the CLI `--scope <PATH>` is a required clap field: not `Option`, no default, no env var.

The check runs on the emission path, not only at the adapter. `manifest.allows(target)` is re-checked immediately before every dispatch, and again before service detection's second connection; four tests assert *zero accepts on a live `TcpListener`* for a denied target, a mismatched scope, an expired manifest and that second connection — real counters, not the scanner's own ledger. One out-of-scope address refuses the whole scan, not the address.

Scans wider than 64 addresses — a `/26`; the default, and server configuration with no spelling in any schema — return `input_required` with an `elicitation/create` and start nothing, so a routine `/24` sweep blocks on a human. `skip_approval` and `auto_approve` fail to parse rather than being rejected. The resume token is HMAC-sealed, argument-bound, TTL-limited and single-use: an approval for one scan cannot start a wider one, and a declined answer starts nothing however valid the token.

Limits. v0.1 records a manifest signature but does not verify it: the file is the authorization, and whoever can write it can authorize a scan — **including this agent, if the same loop holds a file-write tool.** The design assumes the manifest lives where the loop cannot write. And the shipped connect path checks scope *twice* before every packet, not three times: the third check is `bathy-packetd`'s own, deciding from its own `Init` and importing nothing from `bathy-scope`, but no shipped surface spawns that daemon in v0.1, so it guards a SYN path no user can reach. Stdio carries no authenticated principal, so a host that auto-answers the elicitation satisfies the gate, and the principal bound is the constant `"\0unidentified"`. The CLI builds no approval gate at all, deliberately, with a test pinning the exemption. Scope is exempt nowhere. → tests/mcp.rs:1137,2841,3057,3086,3114,3260 · cli.rs:204,452 · scheduler.rs:984,1409,3979,4133,4194,5006 · mcp/config.rs:11-15 · mcp/approval.rs:69,197-278 · scope/manifest.rs:49-57 · gates.rs:3874 with xtask check-packetd · the two scope implementations are cross-checked by proptest at 1024 cases in CI (syn.rs:1539) and at PROPTEST_CASES=310000 cargo test -p bathy-packetd --lib the_two_scope_implementations_agree_on_every_generated_address, zero disagreements, bathy-scope a dev-dependency

3. Every service finding cites the bytes it came from

`evidence_refs` on `service.observed` and `host.discovered` is `NonEmpty<Digest>`: private inner `Vec`, `Deserialize` routed through a rejecting `TryFrom`. An evidence-free finding is not a representable value, and a stored document with `"evidence_refs": []` fails to parse. The published contract carries the same constraint: `schemas/event.json` resolves the field to `minItems: 1`, and `check-schemas` regenerates all 27 schemas from the types, failing on any difference. Evidence is stored before the event citing it is appended; `evidence.get` re-hashes on read and returns `Corrupt` rather than bytes on a mismatch, and an end-to-end test asserts it returns byte-for-byte what the peer sent. `fingerprint.explain` resolves the rule a finding names to a rationale and a source: `bathy --json explain http.server.nginx.v1` returns RFC 9112 §4, RFC 9110 §10.2.4, and a capture from `nginx:1.27-alpine sha256:65645c7b…`.

Precisions. The port event's wire name is `port.state` (`PortStateObserved` in Rust) and its evidence is `Option`, so a port record with no service finding still serialises `"evidence_refs": []` — one does, at 10.30.0.13:33060, in the very document tabled below. The guarantee is on service and host findings, not the field name. The matched byte range is checked but not published: capped evidence could leave the span past what `evidence.get` returns.
→ nonempty.rs:43-120 · event.rs:220,223,229,433 · scheduler.rs:481-490,1493,4877 · evidence/src/store.rs:286 · end_to_end_scan.rs:333 · interpret/src/rules.rs:822-836

WHAT THE OUTPUT COSTS TO CONSUME

Runs of 2026-08-09, same lab: nmap at 23:24:43 (whole scan) and 23:30:00 (--open); bathy twice, keys agentcost-sv, agentcost-po. tiktoken 0.12.0 cl100k_base, a GPT-family encoder — ratios load-bearing, absolutes not. Artifacts sit in target/agentcost/, kept out of the repository by .gitignore:1; measure.sh there produces every row but the --open pair, run by hand with --open and --state open.

CLI STDOUT — 11 ADDRESSES × 13 PORTS, 12 OPEN ENDPOINTS	NMAP B / TOK	BATHY B / TOK	RATIO B / TOK
`nmap -n -Pn -sT -sV --open -oX -` vs `bathy result query --state open --json`	13,190 / 5,957	3,749 / 1,283	3.5× / 4.6×
the same pair, nmap's five `servicefp` attributes removed	**7,158 / 2,648**	**3,749 / 1,283**	**1.9× / 2.1×**
no `--open`: all 143 endpoints, 131 of them not open	32,536 / 11,440	23,770 / 6,977	1.4× / 1.6×
ports only, neither side identifying anything	25,404 / 7,635	22,064 / 6,298	1.2× / 1.2×
open + `service=http`, filtered server-side (no nmap equivalent)	—	557 / 190	—

Those rows are CLI stdout. On the MCP wire the same unfiltered result is 23,769 B / 6,977 tok as `structuredContent`, or 51,391 B / 14,923 tok for the whole reply if the client also forwards the equal JSON text mirror, which is a client decision; `tools/list` is 62,835 B / 14,497 tok, once per session. The `--open` result was not captured on that plane. `servicefp` is Nmap's raw service-fingerprint blob, the probe response encoded for submission to its fingerprint database and discardable by a consumer not submitting one: five of them are 45.7% of that document's bytes and 55.5% of its tokens, so 1.9×/2.1× is the headline that survives scrutiny and 4.6× is the one that does not. On the other side, 47% of

bathy's tabled document (1,769 of 3,749 B) is provenance the XML has no equivalent for — `evidence_refs`, `probe_id`, `rule_id`. An agent that never audits a finding should strip it.

WHERE BATHY LOSES

Medians and observed ranges over 5 repetitions, 11 addresses × 13 ports, Linux aarch64 container on the lab bridge, 2026-08-05. bathy 0.1.0-alpha.1, Nmap 7.93, Masscan 1.3.2, RustScan 2.3.0. docs/benchmarks.md:182-272. Unverified: that bench/results.json records the run it claims to. Every figure here is 143 address-port pairs and 12 open ports; nothing on this page establishes how bathy behaves above a /24, where concurrency, pacing and the 2,000 ms per-connection timeout (scheduler.rs:365) decide the answer.

OPERATION	BATHY MS	AGAINST	MS	BATHY IS
port discovery only	2,105 (993–2,353)	nmap -sT	1,473 (1,467–2,376)	1.4× slower
port discovery only	2,105 (993–2,353)	nmap -sS	1,355 (1,320–1,389)	1.6× slower
port discovery only	2,105 (993–2,353)	rustscan	2,053 (1,013–2,076)	1.0×
service detection (the default)	26,315 (26,303–38,025)	nmap -sV	17,790 (17,784–17,952)	1.5× slower

Not like-for-like, and published anyway: against bare port sweeps that identify nothing, bathy's identifying default is 17.9× `nmap -sT`, 19.4× `nmap -sS`, 12.8× rustscan and 7.1× masscan. The informative numbers are 1.4× on identical work and 1.5× on identification, where detection costs bathy about 24 s and Nmap about 16 over twelve open ports — and bathy's are the noisy timings here: at the top of its range, 38.0 s against 17.95 s, that gap is 2.1×. The kernel's `Tcp: ActiveOpens` counter corroborates those rows from outside every scanner's self-report: bathy-ports-only and rustscan each open exactly 143 connections for the plan's 143 address-port pairs; `nmap -sT` opens 170, `-sV` 294, both SYN rows 0.

- **Coverage is a fraction of Nmap's.** Eight protocols and thirteen interpretation rules against 28 years of community fingerprints. Scored against the six products the lab establishes, the two identifying tools tie at 5/6 products and 3/4 versions; as a raw census over the tabled run's twelve open endpoints, Nmap named a product on 8 and a version on 6, bathy on 6 and 4. The lab was built to exercise exactly those thirteen rules, so it flatters bathy and the gap widens elsewhere. → `docs/benchmarks.md:257-258,329-330,350-356; target/agentcost/nmap-version-open.xml, bathy-query-open.json`
- **bathy identifies nothing behind TLS.** On any TLS-wrapped port the answer is `tls` and stops — three of the twelve open endpoints in that run (10.30.0.16:443, :853, 10.30.0.17:443), a quarter of the result, and far more on a network where TLS is the norm. The cause is scheduler policy, not a missing rule: `detect_service` returns on the first probe that interprets to anything, so `tls-v1` wins and the HTTP probe never runs, while the nginx rule matches those exact bytes under test. Per endpoint, Nmap names a product at three the tabled bathy document does not (PostgreSQL :5432, Redis :6379, nginx :443 on .17); bathy names MySQL at 10.30.0.13:3306 where Nmap does not. Against ground truth only nginx is a loss — the lab records `product: null` at 5432 and 6379, neither service volunteering bytes — and it holds that endpoint to being unidentified, so the day bathy closes it the conformance suite fails until the concession is deleted. → `scheduler.rs:1452-1461; rules.rs:1344; lab/ground-truth.json:31,48,75-77; docs/benchmarks.md:344`
- **Whole capabilities are absent, not slower.** No UDP, no OS detection, no scripting engine, no host discovery, no `-oG`/`-oN`/`-oX`; no IPv6, which is refused rather than unimplemented. And every scan needs a manifest an operator wrote first: there is no ad-hoc mode, and one typo in a target list refuses the whole run rather than the address.
- **The typed surface is a real per-session tax.** 14,497 tokens for `tools/list` against about 4,674 saved per query — but only per fresh scan result the agent has to read. An agent that scans once and asks one in-context nmap XML ten questions pays 5,957 once, and never reaches break-even. bathy's case is the long session over changing state, not one scan interrogated repeatedly.
- **Accuracy distinguished nothing.** All seven runs from all four scanners: 12/12 known-open ports, 0 false negatives, 0 false positives — on a container bridge with no loss or latency. RustScan's repetitions were bimodal, so the 1.0× row is soft, and `nmap -sV` is bimodal across executions: the other taken that day gives 1.1× rather than 1.5×. This is the project's own benchmark of its competitors — one machine, one of two executions committed.

`docs/benchmarks.md`'s loss list is generated from the results by code and guarded by two CI detectors: editing bathy's median from 2,105 to a flattering 1,105 ms exits 1, and deleting the identification-loss bullet exits 1 with a second violation naming the omitted category, both verified by mutating the live document. This page is hand-written from that source and under no gate: `grep -rn why-bathy xtask/src/` is empty. → `xtask/src/bench.rs:58,886; cargo run -p xtask -- check-bench`

WHO THIS IS FOR

Reach for Nmap

- You need breadth of identification, or anything at all behind TLS. Thirteen rules is thirteen rules.
- You need half-open scanning: v0.1 exposes no SYN path to any user, and a CI gate proves neither production scheduler site reaches the privileged daemon. → `gates.rs:3874; xtask check-packetd`
- You need IPv6, refused outright in v0.1 rather than unimplemented, or Windows without WSL2. Linux x86-64/aarch64 is the tested target; macOS builds and passes, but the Docker lab is unreachable from a Mac host. → `README.md:291,340; docs/platform-support.md:5-8`
- You are doing red-team work that must model an attacker's traffic. That is a real need and bathy is the wrong instrument for it: every HTTP probe sends a fixed `User-Agent`, every SMTP probe a fixed `EHLO bathy.invalid`, compile-time constants with no flag to remove them. Evasion and anonymization are permanent non-goals *here* — not a judgement about tools that implement them.
  → `SECURITY.md:112-118,157-162`

Reach for bathy

- A model authors the call and consumes the result, and you want the call to be un-authorable as a command string.
- The authorization has to be a document an operator wrote, which the loop cannot supply, widen or exempt itself from through the tool surface — checked twice before every packet on the shipped connect path, and a third time in the privileged daemon v0.1 does not expose.
- You need to answer "which bytes justified this?" from a digest the finding already carries, without a second scan.
- A long-lived connection that queries results repeatedly — one `tools/list` to a session. One scan and one look, and the 14,497-token schema tax is pure loss.

TRY IT

bathy is for scanning networks you are authorized to scan. Scanning without authorization may be unlawful in your jurisdiction and may violate your provider's terms of service — that is your responsibility, not the tool's. → `README.md:11-12,27-29`

```
cargo install bathy --version 0.1.0-alpha.2
```

Then write a scope manifest and pick a non-loopback address on an interface you own: `127.0.0.0/8` is refused on every platform, so localhost is not the smoke test. README's quickstart is six steps; step 2 cannot be worked around. `bathy serve mcp` implements MCP revision `2026-07-28`, which dropped the `initialize` handshake — not what most clients speak. A Legacy client that still sends one is answered with the version this server implements, and its next `tools/list`, carrying no `_meta`, returns all eleven tools; naming a different version in `_meta` gets `-32022` with `data.supported`.
→ `README.md:90-98,109-114,328-331 · tests/mcp.rs:984 · bathy-mcp/src/server.rs:57,165 · docs/protocol-notes.md:12-19`

`crates.io/crates/bathy` · `github.com/russell0/bathy` · Apache-2.0 OR MIT · alpha.2 published 2026-08-09. Every figure is measured, or asserted by a named test or a printed command, at HEAD 33dce4d. Two fabricated RFC quotations shipped in this repository and were caught; `CONTRIBUTING.md:56-61` now requires verbatim quotation from three checkable artifact kinds, pinned by `check-docs`. Clean room: no Nmap data file was read.